

A SimPy model

Your model consists of a source that generates customers randomly. Each customer waits for a counter (resource) to become available. The customer then spends a random amount of time to get his work done before leaving. We'll include the necessary packages and write the code for generating customers:

```
import SimPy.Simulation as simpy
from scipy.stats import erlang
class Source(simpy.Process):
    """Generate customers randomly"""
    def generate(self, number, resource, pdf, pdf_counter):
        for i in range(number):
            c = Customer(name="Customer%02d"%(i+1))
            simpy.activate(c, c.visit(pdf=pdf, counter=resource))
            # wait for the next customer
            t = pdf.rvs()
            yield simpy.hold, self, t
```

The class *Source* contains one method—*generate*. It takes the following as parameters: the number of customers to be generated; counter resources and probability distribution functions for the arrival of customers; and the service time at the counter. You create a customer and activate the customer by passing a method name (*visit*, in your case) as a parameter. The simulation package will call the visit method, which will need to be defined in the *Customer* class. Then, you find the random time at which the next customer is expected and hold the process till then.

You may now define the *Customer* class as follows:

```
class Customer(simpy.Process):
    def visit(self, pdf, counter):
        time_in = simpy.now()
        yield simpy.request, self, counter
        wait_time = simpy.now() - time_in
        tlb = pdf.rvs()
        yield simpy.hold, self, tlb
        yield simpy.release, self, counter
        time_in_bank = simpy.now() - time_in
        process(self.name, time_in, wait_time, time_in_bank)
```

The *Customer* class contains the method *visit*, which encompasses the logic for the interaction of the customer with the bank. You track the time the customer comes in and as he waits for a counter to become available, you compute the wait time. Then, you find the random time for servicing this customer and hold the process till it gets over. After this, you release the counter. Finally, you'll compute the time spent by the customer in the bank and process the times.

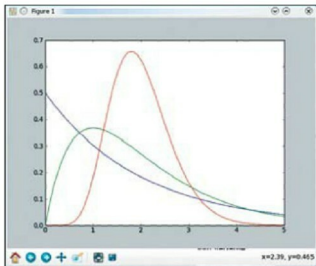


Figure 1: Erlang distribution probability functions with mean 2: Shape=1 (Blue), Shape=2 (Green), Shape=10 (Red)

```
def main(maxTime, maxNumber, arrival_interval, mean_serve_time, counters=1):
    print("1 counters with mean arrival and service times %4.1f %4.1f minutes"
          %(counters, arrival_interval, mean_serve_time))
    k = simpy.Resource(capacity=counters, name="Counter", unitName="Clerk")
    simpy.initialize()
    s = Source("Source")
    simpy.activate(s, s.generate(number=maxNumber,
                                pdf=erlang(1, scale=arrival_interval),
                                pdf_counter=erlang(2, scale=mean_serve_time/2.0),
                                resource=k, at=0.0))
    simpy.simulate(until=maxTime)
```

Your main program starts the simulation by creating the necessary resources, and activating the source process for generating customers using the *generate* method defined in the *Source* class. You use the Erlang distributions with shape factor 1 and 2 for arrivals and serving time.

```
def process(name, time_in, wait_time, time_in_bank):
    print("%s %6.2f %6.2f %6.2f"%(name, time_in, wait_time, time_in_bank))

main(100, 5, 1.5, 2.0, 1)
```

For processing, just print the results and run the simulation with a customer arriving every 1.5 minutes and needing an average of 2 minutes to be served. You would expect the queue to build up pretty quickly as you can see from the results:

```
1 counters with mean arrival and service times 1.5 2.0 minutes
Customer00 0.00 0.00 1.08
Customer01 0.36 0.72 3.21
```